به نام خدا

سیستم‌های بی‌درنگ
فاز یک پروژه

سبحان آقاسی زاده ۴۰۲۱۰۵۶۶۵ - پارمیس حمصیان ۴۰۲۱۷۰۹۱۳

۱. ۱. هدف پروژه

هدف پروژه طراحی یک چارچوب مبتنی بر SDN برای شبکه‌های محاسبات لبه خودرویی است. در این شبکه، ماشین‌ها وظیفه‌های محاسباتی تولید می‌کنند و هر وظیفه می‌تواند روی خود ماشین، روی ماشین ارائه‌دهنده، یا روی سرور edge پردازش شود. هدف کلی این است که تاخیر، گم‌شدن بسته‌ها، تعداد retransmission و تعداد نقطه کور کمتر شود.

در فاز دوم، پیاده‌سازی کامل‌تر پروژه انجام شد. فاز اول به عنوان baseline نگه داشته شد و سپس روش SDN/PPO/Kalman برای جایگذاری ماشین‌های ارائه‌دهنده و بارسپاری وظیفه اضافه شد.

۲. سناریوهای شبیه‌سازی

دو سناریوی خواسته‌شده در صورت پروژه پیاده‌سازی شدند:

سناریو	تعداد خودروها	تعداد ماشین‌های ارائه‌دهنده	تعداد سرورهای لبه
سناریوی ۱	۲۰	۳	۲
سناریوی ۲	۵۰	۶	۴

هر سناریو هم با روش baseline و هم با روش فاز دوم اجرا می‌شود تا مقایسه مستقیم داشته باشیم.

۳. اجزای اصلی پیاده‌سازی

کد پروژه به چند فایل ساده تقسیم شده است:

```
phase1/config.py       scenario and simulation parameters
phase1/models.py       dataclasses for vehicles, tasks, links, metrics
phase1/environment.py  mobility, weather, traffic, packet-loss model
phase1/placement.py    random provider-vehicle placement
phase1/offloading.py   task-offloading and route-selection algorithm
phase1/simulation.py   simulation loop and CSV writers
phase2/kalman.py       Kalman link-quality estimation
phase2/sdn_controller.py PPO-style SDN placement controller
phase2/runner.py       training, evaluation, comparison
```

```
phase2/plotting.py     SVG plots
run_phase1.py          command-line runner
run_phase2.py          full Phase 2 runner
```

این تقسیم‌بندی باعث می‌شود روش پایه و روش فاز دوم جدا ولی قابل مقایسه باشند. فایل‌های phase1 محیط و baseline را نگه می‌دارند و فایل‌های phase2 بخش Kalman، کنترل‌کننده SDN و نمودارها را اضافه می‌کنند.

۴. مدل محیط

در هر گام زمانی، هر ماشین ممکن است یک وظیفه جدید تولید کند. هر وظیفه سه ویژگی اصلی دارد:

- اندازه داده ورودی بر حسب مگابیت
- حجم پردازش مورد نیاز بر حسب میلیارد سیکل
- مهلت زمانی انجام وظیفه

برای مدل‌سازی شرایط شبکه، چند عامل ساده در نظر گرفته شده است:

- فاصله بین فرستنده و گیرنده
- تراکم ترافیک محلی
- وضعیت آب‌وهوا

اگر فاصله دو گره از برد بیشتر باشد، آن لینک نامعتبر است. اگر لینک معتبر باشد، احتمال گم‌شدن بسته با این رابطه ساده حساب می‌شود:

```
loss = base_loss + distance_effect + traffic_effect + weather_penalty
```

در فاز دوم، مقدار خام گم‌شدن بسته به عنوان اندازه‌گیری noisy وارد فیلتر کالمن می‌شود و خروجی فیلتر برای تصمیم بارسپاری استفاده می‌شود.

۵. جایگذاری ماشین‌های ارائه‌دهنده

در روش پایه، جایگذاری ماشین‌های ارائه‌دهنده به صورت رندوم و یکنواخت در محیط دوبعدی انجام می‌شود. در فاز دوم، کنترل‌کننده SDN هر چند گام یک بار مکان ماشین‌های ارائه‌دهنده را به‌روزرسانی می‌کند.

state کنترل‌کننده شامل این موارد است:

```
demand center
weather loss penalty
average Kalman packet-loss estimate
provider queue pressure
```

action کنترل‌کننده انتخاب یک strategy برای جایگذاری است. هر strategy مختصات پیوسته برای ماشین‌های ارائه‌دهنده تولید می‌کند. به‌روزرسانی policy با یک روش سبک و قابل توضیح شبیه PPO انجام می‌شود.

۶. الگوریتم بارسپاری وظیفه

برای هر وظیفه، الگوریتم همه انتخاب‌های ممکن را بررسی می‌کند:

- پردازش محلی روی خود ماشین
- ارسال مستقیم به ماشین ارائه‌دهنده
- ارسال مستقیم به سرور edge
- ارسال به سرور edge از طریق ماشین ارائه‌دهنده

گزینه‌هایی که خارج از برد باشند حذف می‌شوند. این همان action masking در نسخه ساده فاز اول است.

برای هر گزینه معتبر، این مقادیر محاسبه می‌شود:

- تاخیر ارسال
- تاخیر پردازش
- احتمال گم‌شدن بسته
- تعداد مورد انتظار retransmission
- اینکه deadline از دست می‌رود یا نه

سپس برای هر گزینه یک امتیاز محاسبه می‌شود:

```
score = delay + loss_weight * packet_loss + deadline_penalty
```

گزینه‌ای که کمترین score را داشته باشد انتخاب می‌شود. در روش فاز دوم، مقدار packet loss داخل این score از خروجی فیلتر کالمن می‌آید.

۷. تابع reward

برای آموزش کنترل‌کننده SDN، reward از روی معیارهای اصلی پروژه ساخته شده است:

```
reward = -(delay cost + packet-loss cost
           + deadline cost + blind-spot cost)
```

پس اگر تاخیر، گم‌شدن بسته، deadlineهای از دست‌رفته و نقطه کور کمتر شوند، reward بهتر می‌شود. برای جلوگیری از تغییر شدید policy، از clipping شبیه PPO استفاده شده است.

۸. معیارهای خروجی

معیارهای زیر ذخیره می‌شوند:

- میانگین نسبت گم‌شدن بسته
- میانگین تاخیر سیستم
- تعداد وظیفه‌هایی که deadline را از دست داده‌اند

- تعداد رخدادهای نقطه کور
- میانگین موقعیت ماشین‌های ارائه‌دهنده
- تعداد انتخاب هر مقصد و هر مسیر

خروجی‌های فاز دوم در پوشه `outputs/phase2` ذخیره می‌شوند. فایل `summary.csv` مقایسه روش پایه و روش SDN/PPO/Kalman را دارد. فایل `convergence.csv` برای نمودار همگرایی RL است. نمودارهای نهایی هم در پوشه `outputs/phase2/plots` تولید می‌شوند.

۹. اجرای برنامه

پس از اجرای پیش‌فرض برنامه با دستور زیر:

```
python3 run_phase2.py
```

برنامه ابتدا baseline را اجرا می‌کند، سپس کنترل‌کننده SDN/PPO را آموزش می‌دهد، بعد روش فاز دوم را ارزیابی می‌کند و در پایان فایل‌های CSV و نمودارهای SVG را می‌سازد.

خلاصه اجرای نهایی به شکل زیر است:

سناریو	روش	میانگین loss	میانگین delay	deadline از دست‌رفته	نقطه کور
۱	baseline	۰۹۸۸.۰	۴۰۵۳.۰	۳۲	۱۶۲
۱	SDN/PPO/Kalman	۰۵۹۲.۰	۴۴۷۷.۰	۱۸	۹۲
۲	baseline	۰۹۶۸.۰	۳۴۸۰.۰	۳۷	۶۷
۲	SDN/PPO/Kalman	۰۵۹۶.۰	۴۲۲۱.۰	۳۵	۹

نتیجه نشان می‌دهد روش فاز دوم در هر دو سناریو گم‌شدن بسته و نقطه کور را کمتر کرده است. تعداد deadlineهای از دست‌رفته هم بهتر شده است. میانگین delay کمی بیشتر شده، چون الگوریتم فاز دوم نسبت به لینک‌های پرریسک محافظه‌کارتر عمل می‌کند و برای کاهش packet loss بعضی هاtask را محلی‌تر یا از مسیرهای امن‌تر پردازش می‌کند.

۱۰. جمع‌بندی

در پیاده‌سازی نهایی، روش رندوم فاز اول به عنوان baseline حفظ شد. سپس فیلتر کالمن برای تخمین کیفیت لینک، کنترل‌کننده SDN برای جایگذاری ماشین‌های ارائه‌دهنده، به‌روزرسانی سبک PPO برای یادگیری هایstrategy جایگذاری، و نمودارهای نهایی اضافه شدند. ساختار پروژه ساده نگه داشته شده تا تصمیم‌ها قابل توضیح باشند و مقایسه با روش پایه روشن باشد.